

基于光滑化策略与梯度类算法的信号重构问题研究

摘 要

信号重构是信号处理的核心问题：图像在采集与传输中常被椒盐噪声污染，压缩感知中需要由少量线性观测恢复高维稀疏信号，二者在数学上均可归结为带 ℓ_1 型非光滑项的凸优化模型，其求解质量取决于模型的建立与数值算法的设计。本文围绕“非光滑问题的光滑化处理、凸模型的盒约束化、投影梯度与共轭梯度两类算法设计”这一主线，对三个问题建立统一的建模求解框架，并给出完整的数值检验。

针对问题一，首先，按“取值为动态范围端点且被滤波器替换”的两条件定义确定噪声候选集，将误检率控制在 1% 以内；其次，在候选集上建立含 ℓ_1 数据保真项与保边正则项的凸泛函，利用势函数的偶对称性将正则项改写为仅依赖四邻域查值的边和形式；再次，以平方根型光滑函数逼近绝对值函数，并由最大值原理把模型约束到像素动态范围，得到盒约束光滑凸问题；最后，设计 BB 步长的投影梯度算法求解。在 12 张标准测试图、2 个噪声等级、3 组随机种子共 72 个算例上全部收敛，平均恢复 PSNR 为 37.33 dB 与 33.76 dB；Lena 图在 30% 噪声下达到 38.35 dB，较自适应中值滤波直接输出提高 4.47 dB。

针对问题二，将五种保边势函数置于同一恢复框架下，先逐一分析其光滑性、导数有界性与原点曲率等解析性质，再按“各自独立标定参数、统一条件下做参数迁移验证”的协议比较。结果表明：各势函数在标定参数下恢复质量接近（PSNR 极差 0.19 dB 与 0.13 dB），差异主要体现在参数敏感性与收敛速度。平方根型的参数平台较宽且收敛最快；光滑化幂函数型的恢复质量与效率也较稳定；对数余弦型、对数型和 Huber 型在本文求解配置下需要更多迭代。据此优先推荐平方根型或光滑化幂函数型。

针对问题三，将光滑化策略迁移到 ℓ_1 正则化最小二乘，配合光滑参数逐级续延，用修正 PRP 共轭梯度法求解，并与变量分裂框架下的 GPSR-BB 投影梯度法在完全一致的实例上对比。在 4096 维、160 尖峰实例上，两法均完整恢复全部非零元；按“同目标值”口径修正 PRP 的迭代数为 GPSR 的 4.2 倍，去偏置后相对误差 0.0286 对 0.0265，恢复质量基本持平；共轭参数对照显示修正 PRP 与 HS 参数的迭代数不到 FR 与 DY 参数的四分之一。

关键词：两阶段图像恢复； ℓ_1 光滑化；保边势函数；投影梯度算法；共轭梯度法

一、 问题重述

1.1 问题背景

信号处理是一门重要的交叉学科，在通信、图像处理、生物医学工程、雷达技术等领域发挥着关键作用。图像恢复是其中的经典问题：图像可能因传输过程中的噪声、压缩、失真、模糊等原因受损，需要从受损观测中重建清晰图像。椒盐噪声是最具代表性的脉冲噪声，受污染像素的灰度值退化为动态范围的两个端点，其真实值完全丢失，因此恢复工作分为“检测哪些像素被污染”与“恢复被污染像素”两个阶段 [2]。稀疏信号重构则是压缩感知的核心问题：利用远少于信号维数的线性观测恢复稀疏信号，可归结为 ℓ_1 正则化最小二乘模型 [1]。

两类问题的共同数学本质是带有 ℓ_1 型非光滑项的凸优化：目标函数中的绝对值项不可微，常用梯度类算法无法直接使用。光滑化策略 [4, 9] 用一族误差受控的光滑函数逼近绝对值函数，把非光滑问题化为光滑问题，是处理此类问题的通用技巧。本文即以光滑化为主线，配合盒约束化技术与梯度类优化算法，系统研究题目给出的三个问题。

1.2 问题提出

问题一：采用光滑化策略建立图像恢复问题的优化模型，对噪声等级为 30% 与 50% 的 salt-and-pepper 噪声模型，使用投影梯度算法求解，并构建所需迭代次数、消耗时间、信噪比 SNR、峰值信噪比 PSNR 等评价指标。

问题二：图像恢复模型中涉及保边势函数 φ_α ，题目给出五种候选，要求探索保边势函数的不同选择对图像恢复效果的影响。

问题三：采用修正 PRP 共轭梯度算法，重构长度为 2^{12} 且包含 160 个随机产生的 ± 1 非零元素的稀疏信号，并与 GPSR 算法进行对比。

二、 问题分析

2.1 问题一的分析

问题一是“检测 + 正则化恢复”两阶段框架下的非光滑凸优化问题。首先，受污染像素取值恰为动态范围端点，故检测的关键是在保真像素与污染像素之间建立可靠判决：自适应中值滤波通过逐步扩大窗口直至窗口内出现正常明暗层级，保证在高噪声下仍能利用足够多的保真像素作出判断。其次，恢复目标既要清除噪声又要保持边缘纹理，需要在数据保真项与邻域正则项之间权衡；其中 ℓ_1 数据项不可微，这是求解的主要障碍。接着，处理非光滑性的通用技巧是光滑化，即用一族误差受控的光滑函数逼近绝对值函数；光滑化后问题成为光滑凸问题，再利用最大值原理把求解域限制到像素动态范

围内，即得盒约束光滑凸问题。最后，采用 GPSR 投影梯度算法求解该盒约束光滑凸问题，并沿 BB 步长框架设计求解器，并构建迭代数、时间、SNR、PSNR 等指标进行评价。

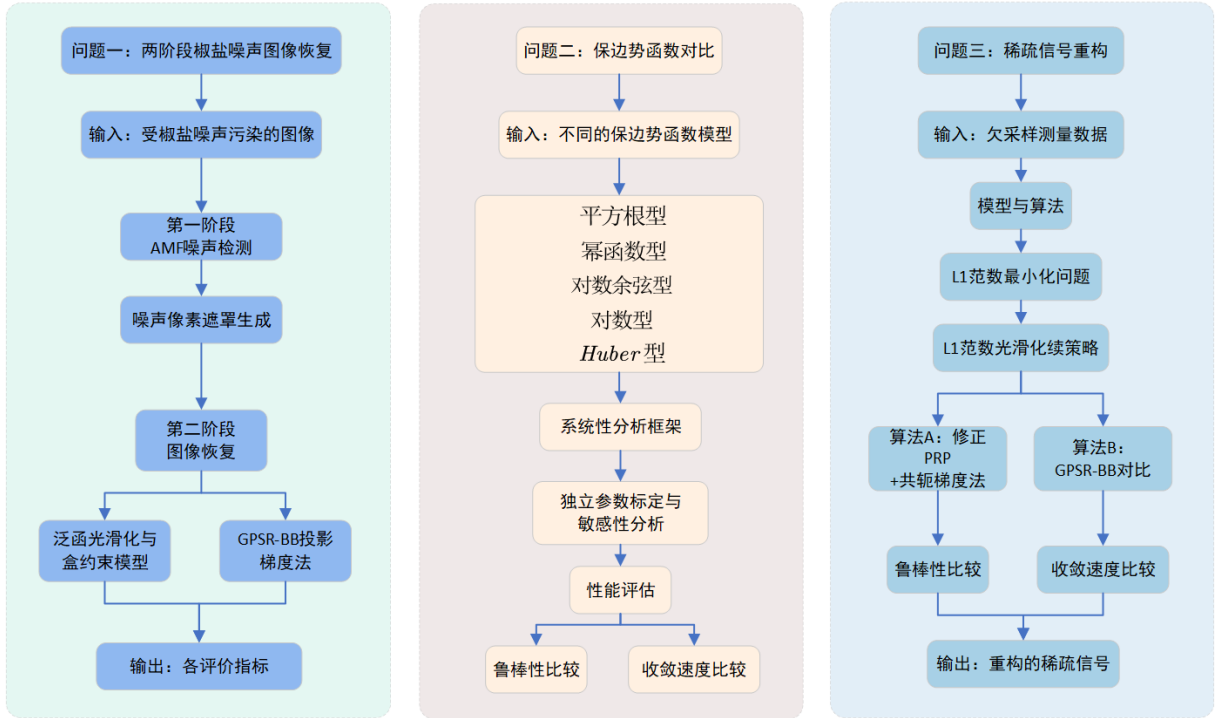


图 1 全文研究框架与三个问题的求解流程

2.2 问题二的分析

问题二要求比较五种保边势函数对图像恢复结果的影响。本文沿用问题一的两阶段方法：第一阶段仍使用 AMF 检测噪声位置，第二阶段保持数据保真项、邻域结构和求解算法不变，仅替换正则项中的势函数。由于五种势函数的参数含义和数值尺度不同，本文分别标定其参数，并在相同图像、噪声条件和停止准则下比较恢复质量与求解效率，最后确定较优的势函数。

2.3 问题三的分析

问题三是大规模稀疏信号重构，其模型为 ℓ_1 正则化最小二乘，同样非光滑。首先，通过变量分裂把模型化为有界约束二次规划 [1] 后用投影梯度求解，形成第一条技术路线；本文则沿用问题一的光滑化路线，把 ℓ_1 项光滑化后用非线性共轭梯度法求解，形成第二条技术路线，二者构成正面对比。其次，修正 PRP 共轭参数采用非负截断，并配合强 Wolfe 线搜索与下降性重启；只有在水平集有界、梯度 Lipschitz 连续且线搜索等条件成立时，才可引用相应的全局收敛结论 [10]，不能把非负截断本身等同于收敛保证。最后，两算法的停止准则与迭代机制不同，比较时须采用“达到相同目标值”的口径，并对 ℓ_1 解的幅度收缩做去偏置处理，比较才有意义。

三个问题的总体研究框架与求解流程如图 1 所示。

三、 模型假设

1. 假设图像中的椒盐噪声为相互独立的随机过程，每个像素受污染与否与其位置和取值无关，且椒、盐出现的概率相等；
2. 假设图像经噪声破坏后其余像素值保持不变，动态范围端点值在图像中仅可能来自噪声或真实灰度极值；
3. 假设恢复过程中图像的结构性质可用相邻像素差值的灰度分布刻画，即保边势函数模型对自然图像成立；
4. 假设稀疏信号重构中测量矩阵的行经过正交化处理、测量噪声为高斯白噪声，且噪声方差已知；
5. 假设数值求解时浮点误差与图像量化误差均可忽略，所有评价指标在未取整的浮点恢复图上计算；
6. 假设算法性能指标（迭代数、运行时间与恢复误差）在一定随机种子范围内具有代表性，各实验重复多次并取均值。

四、 符号说明

符号	说明	单
A	图像像素索引集 $\{1, \dots, M\} \times \{1, \dots, N\}$ （问题三中 A 为测量矩阵，按所在问题区分）	
$y_{i,j}$	观测图像在位置 (i, j) 的灰度值（问题三中观测向量为 b ）	灰
u_p	候选噪声像素 p 的恢复值（决策变量）	灰
x_i	待重构稀疏信号的第 i 个分量	
$F(\cdot)$	目标泛函（目标函数）	
φ_α	带参数 α 的保边势函数	
ρ_μ	带光滑参数 μ 的 ℓ_1 光滑逼近函数	
β	正则项权重，平衡数据保真与平滑	
$P(\cdot)$	到可行域（盒约束或非负象限）的欧氏投影	
g_k, d_k	第 k 步迭代的梯度向量与搜索方向	
α_k, β_k	第 k 步的线搜索步长与共轭参数	
$\ \cdot\ $	向量范数（ ℓ_1 、 ℓ_2 或 ℓ_∞ ，视上下文）	

其余仅在单一问题中使用的符号（如噪声候选集 Ω 、测量矩阵 A 等）在其首次出现处随公式逐一说明。

五、 问题一模型的建立与求解

5.1 噪声模型与两阶段总体框架

记原始图像为 X ，其像素索引集为 $A = \{1, \dots, M\} \times \{1, \dots, N\}$ ， $M \times N$ 为图像尺寸；像素动态范围为 $[s_{\min}, s_{\max}]$ ，8 位灰度图取 $s_{\min} = 0$ 、 $s_{\max} = 255$ 。经典 salt-and-pepper（椒盐）噪声模型下，观测像素为

$$y_{i,j} = \begin{cases} s_{\min}, & \text{以概率 } p, \\ s_{\max}, & \text{以概率 } q, \\ x_{i,j}, & \text{其余,} \end{cases} \quad (1)$$

其中 $x_{i,j}$ 为原始像素值， $r = p + q$ 称为噪声等级。本文取 $p = q = r/2$ 。为直观说明椒盐噪声的生成过程，以 Lena 灰度图像为例。对每个像素独立生成随机数 $\xi_{i,j} \sim U(0, 1)$ ：当 $\xi_{i,j} < p$ 时，将像素置为 $s_{\min} = 0$ ；当 $\xi_{i,j} \geq 1 - q$ 时，将像素置为 $s_{\max} = 255$ ；其余情

况下保持原像素值不变。图 2 给出了一个 3×3 局部区域的加噪过程。

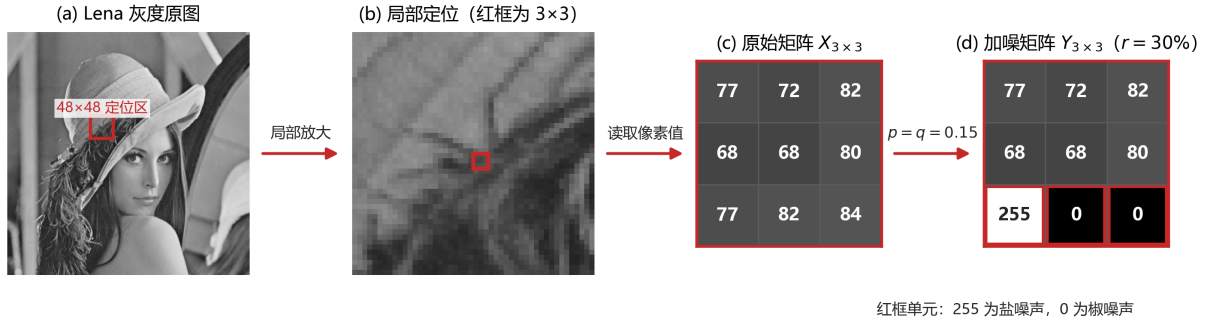


图 2 Lena 图像的椒盐噪声生成过程 ($r = 30\%$)

由于受污染像素的真实值完全丢失, 无法对单个像素单独修复, 因此采用两阶段方法: 第一阶段检测出噪声像素的候选集 Ω 并给出初值, 第二阶段仅在 Ω 上通过最小化一个保边正则化泛函恢复像素。下面依次建立两个阶段的模型。

5.2 第一阶段: 自适应中值滤波检测

中值滤波是最常用的脉冲噪声滤波器: 它把每个像素替换为其邻域内像素灰度的中位数。普通中值滤波对全图统一处理, 会在去除噪声的同时损伤细节。自适应中值滤波 (Adaptive Median Filter, AMF) [2] 的改进在于两点: 其一, 只对疑似噪声像素做替换; 其二, 当窗口内没有足够的正常灰度层次时自动扩大窗口。

对奇数窗口边长 w , 记半径 $h = (w-1)/2$, 并定义 $S_{i,j}^w = \{(k,l) : |k-i| \leq h, |l-j| \leq h\}$; 当 $(k,l) \notin A$ 时, 其灰度值按下文的反射规则由边界内像素补齐。于是 $S_{i,j}^w$ 表示中心在 (i,j) 的 $w \times w$ 窗口, $w_{\max}$ 为允许的最大窗口边长。AMF 的判据来自如下观察: 若窗口内同时存在正常的明暗层次, 即

$$s_{\min}^w < s_{\text{med}}^w < s_{\max}^w, \quad (2)$$

其中 $s_{\min}^w, s_{\text{med}}^w, s_{\max}^w$ 分别为窗口内灰度的最小值、中位数与最大值, 则窗口信息足以作出判决; 否则说明窗口内灰度几乎相同 (噪声成片), 需扩大窗口再试。具体流程为: 对每个像素依次取 $w = 3, 5, 7, \dots, w_{\max}$ 逐级检验上式, 直至满足或达到 $w_{\max}$; 达到上限仍不满足的像素判为噪声并用 $w_{\max}$ 窗口的中值替换。对最终窗口, 用极值判决 $s_{\min}^{w*} < y_{i,j} < s_{\max}^{w*}$ 区分保真像素与噪声像素, 其中 w^* 为该像素停止扩张时的窗口边长。实现中边界像素采用 SciPy ndimage 的 reflect 约定, 即反射时重复端点像素; 这与 NumPy pad 中同名模式的端点约定不同。 $w_{\max}$ 按噪声等级设定: 30% 取 7×7 , 50% 取 9×9 。

结合像素端点取值与 AMF 的替换结果, 定义噪声候选集为 [2]:

$$\Omega = \{(i,j) \in A : y_{i,j} \in \{s_{\min}, s_{\max}\} \text{ 且 } \hat{y}_{i,j} \neq y_{i,j}\}, \quad (3)$$

其中 $\hat{y}_{i,j}$ 为 AMF 的输出。该定义要求候选像素同时满足“取值为动态范围端点”与“被

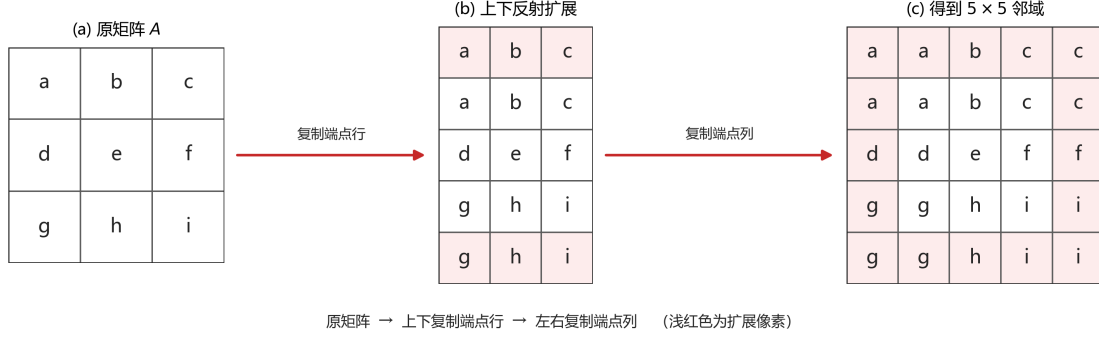


图3 SciPy reflect 约定下的边界像素扩展示意

滤波器替换过”两个条件，可排除两类误判：灰度值恰为端点但被判为保真的像素没有产生替换，不会进入候选集；非端点像素即使被替换也不满足端点条件。经验上该定义使误检率由仅用端点判决时的百分之几降至 1% 以内，是恢复质量提升的重要来源。

5.3 第二阶段：保边正则化恢复模型

第二阶段仅在 Ω 上恢复像素。为使恢复结果“去噪的同时保留边缘”，需要引入正则化的思想。图像的边缘指相邻像素灰度发生跃变的位置，其数学表现是相邻像素差值 $u_p - u_q$ 较大。若用二次函数惩罚邻域差，则跃变处的代价极高，恢复时会把边缘一并抹平；反之，若惩罚函数在差值大时仅按线性增长（如 $|t|$ ），则大差值所受的额外惩罚有限，边缘得以保留。这类“原点光滑、大尺度处近似线性”的偶凸惩罚函数称为保边势函数，记为 $\varphi_\alpha(t)$ ， $\alpha > 0$ 为其参数。问题一主实验取

$$\varphi_\alpha(t) = \sqrt{t^2 + \alpha}, \quad (4)$$

它处处光滑（ C^∞ ）， $t = 0$ 处二阶导 $\varphi_\alpha''(0) = 1/\sqrt{\alpha}$ 有界，且 $|t|$ 很大时 $\varphi_\alpha(t) - |t| \rightarrow 0$ ，兼具平滑性与保边性；其余四种候选势函数在问题二中系统比较。

沿用题目式 (2)，第二阶段的目标函数为

$$\min_u \sum_{(i,j) \in \Omega} \left[|u_{i,j} - y_{i,j}| + \frac{\beta}{2} (S_1 + S_2) \right], \quad (5)$$

其中数据保真项 $|u_{i,j} - y_{i,j}|$ 要求恢复值不远离观测值，

$$S_1 = \sum_{(m,n) \in V_{i,j} \cap \Omega^C} 2\varphi_\alpha(u_{i,j} - y_{m,n}), \quad S_2 = \sum_{(m,n) \in V_{i,j} \cap \Omega} \varphi_\alpha(u_{i,j} - u_{m,n}), \quad (6)$$

其中 $V_{i,j} = \{(i, j-1), (i, j+1), (i-1, j), (i+1, j)\} \cap A$ 为位置 (i, j) 在图像域内的四邻域， Ω^C 为 Ω 在 A 中的补集。 S_1 表示候选像素与干净邻居之间的正则化约束。由于 $(m, n) \in \Omega^C$ ，相邻像素保持观测值 $y_{m,n}$ ； S_2 表示两个候选像素之间的正则化约束，由于 $(m, n) \in \Omega$ ，相邻像素同样是待求变量，因此采用 $u_{m,n}$ 。又因 φ_α 为偶函数，有

$\varphi_\alpha(u - y) = \varphi_\alpha(y - u)$ 。在式 (5) 的外层求和中，候选像素–干净像素边只从候选端出现一次，而候选像素–候选像素边会从两个端点各出现一次。

为进一步看清结构，记 $\tilde{u}_p = u_p$ ($p \in \Omega$)、 $\tilde{u}_p = y_p$ ($p \notin \Omega$)，并定义无向边集合 $E_\Omega = \{\{p, q\} : q \in V_p, \{p, q\} \cap \Omega \neq \emptyset\}$ ，其中每条无向边只保留一次。由 φ_α 的偶性，正则项可等价改写为

$$F_\beta(u) = \sum_{p \in \Omega} |u_p - y_p| + \beta \sum_{\{p, q\} \in E_\Omega} \varphi_\alpha(\tilde{u}_p - \tilde{u}_q). \quad (7)$$

对式 (7) 求梯度，把自身项与来自邻居一侧的贡献合并，并利用 φ'_α 为奇函数，得

$$\nabla F_\beta(u)_p = \text{sign}(u_p - y_p) + \beta \sum_{q \in V_p} \varphi'_\alpha(u_p - \tilde{u}_q). \quad (8)$$

该式的意义在于：梯度只依赖对每个候选像素的四邻域查值，求和规模与 $|\Omega|$ 成正比，不含任何矩阵乘法，为大规模高效求解奠定了基础。

5.4 光滑化策略

式 (8) 中数据保真项的导数 $\text{sign}(\cdot)$ 在原点不存在，目标函数非光滑，标准梯度类算法无法直接使用。处理非光滑优化常用两条路线：一是次梯度类方法，但收敛慢；二是光滑化 [4, 9]，即用一族光滑函数一致逼近非光滑项后求解光滑问题。本文采用后者：选取 C^1 函数 ρ_μ 逼近 $|t|$ ，要求存在与 t 无关的误差界

$$|\rho_\mu(t) - |t|| \leq c_\rho \mu, \quad \forall t \in \mathbb{R}, \quad (9)$$

其中 $\mu > 0$ 为光滑参数， c_ρ 为常数。本文实现三种形式，见表 1。

表 1 三种 ℓ_1 光滑函数及其逼近误差界

名称	$\rho_\mu(t)$	误差界
平方根型	$\sqrt{\mu^2 + t^2}$	μ
Huber 型	$\begin{cases} t^2/(2\mu), & t \leq \mu \\ t - \mu/2, & t > \mu \end{cases}$	$\mu/2$
Softplus 型	$ t + 2\mu \ln(1 + e^{- t /\mu})$	$2\mu \ln 2$

主实验取平方根型。以 ρ_μ 替代式 (7) 中的 $|u_p - y_p|$ 即得光滑化模型

$$\min_{u \in [s_{\min}, s_{\max}]^{|\Omega|}} F_\mu(u) = \sum_{p \in \Omega} \rho_\mu(u_p - y_p) + \beta \sum_{\{p, q\} \in E_\Omega} \varphi_\alpha(\tilde{u}_p - \tilde{u}_q), \quad (10)$$

其中光滑化误差满足 $|F_\mu(u) - F_\beta(u)| \leq c_\rho \mu |\Omega|$ ，即 $\mu \rightarrow 0$ 时目标函数一致收敛到原问题；梯度表达式由式 (8) 把 sign 换成 ρ'_μ 得到。

关于光滑参数 μ 的选取需要权衡： μ 越小逼近精度越高，但平方根型 ρ_μ 在原点附近的最大曲率随 $1/\mu$ 增大，而远离原点时曲率迅速减小，可能使目标函数的条件数变差，从而增加线搜索与迭代的难度； μ 越大光滑越充分，但单个数据项的逼近误差上界为 $c_\rho\mu$ 。消融实验表明，在本文主配置与测试范围内， μ 从 1 降到 10^{-4} 时迭代数与恢复 PSNR 几乎不变。因此主实验取 $\mu = 1$ ，此时平方根型逼近的单项误差不超过一个灰度级；这里的“一致”是数值实验结论，不等同于对 $\mu \rightarrow 0$ 极限的解析证明。

5.5 盒约束化与最优性条件

式 (10) 中的盒约束 $[s_{\min}, s_{\max}]$ 引入是无损的，依据最大值原理 [3]：当 φ_α 为偶函数、连续且关于 $|t|$ 严格递增时，目标泛函的任一全局极小点 u^* 必满足 $u_p^* \in [s_{\min}, s_{\max}]$ 。其直观解释是：把全部分量逐点截断到动态范围内，投影的非扩张性保证相邻像素差的绝对值不会增大，且数据项也不会增大。同时 $\varphi_\alpha'' \geq 0$ 与 $\rho_\mu'' \geq 0$ 保证 F_μ 为凸函数，故式 (10) 是盒约束光滑凸规划，其最优性条件可写为投影残差

$$G(u) = \|P(u - \nabla F_\mu(u)) - u\|_\infty = 0, \quad (11)$$

其中 $P(\cdot)$ 为到 $[s_{\min}, s_{\max}]$ 的逐像素欧氏投影。 $G(u) = 0$ 即一阶 KKT 条件的紧凑表达：在可行域内部梯度为零；在下边界梯度非负，在上边界梯度非正。 $G(u)$ 同时充当算法的终止准则。

5.6 算法设计：GPSR-BB 投影梯度法

GPSR 方法 [1] 原本用于稀疏重构：以变量分裂把 ℓ_1 问题化为有界约束二次规划后，用投影梯度法求解。本文的模型式 (10) 本身已是盒约束光滑凸问题，可直接沿用其框架。投影梯度法是约束优化中梯度法的自然推广：每步先沿负梯度方向移动，再投影回可行域。本文采用其“沿投影弧搜索”的变体：

$$w^{(k)} = P(u^{(k)} - \alpha^{(k)} \nabla F_\mu(u^{(k)})), \quad u^{(k+1)} = u^{(k)} + \lambda^{(k)}(w^{(k)} - u^{(k)}), \quad (12)$$

即从 $u^{(k)}$ 出发到投影点 $w^{(k)}$ 的线段（投影弧）上搜索新迭代点。

步长 $\alpha^{(k)}$ 取 Barzilai–Borwein (BB) 步长：记 $s^{(k)} = u^{(k+1)} - u^{(k)}$ 、 $y^{(k)} = \nabla F(u^{(k+1)}) - \nabla F(u^{(k)})$ ，先定义候选步长

$$\hat{\alpha}^{(k+1)} = \begin{cases} \frac{(s^{(k)})^\top s^{(k)}}{(s^{(k)})^\top y^{(k)}}, & (s^{(k)})^\top y^{(k)} > 0, \\ \alpha_{\max}, & (s^{(k)})^\top y^{(k)} \leq 0. \end{cases}$$

则截断后的 BB 步长写成分段形式

$$\alpha^{(k+1)} = \begin{cases} \alpha_{\min}, & \hat{\alpha}^{(k+1)} < \alpha_{\min}, \\ \hat{\alpha}^{(k+1)}, & \alpha_{\min} \leq \hat{\alpha}^{(k+1)} \leq \alpha_{\max}, \\ \alpha_{\max}, & \hat{\alpha}^{(k+1)} > \alpha_{\max}, \end{cases} \quad (13)$$

其中 $[\alpha_{\min}, \alpha_{\max}] = [10^{-10}, 10^6]$ ；曲率信息失效 ($(s^{(k)})^\top y^{(k)} \leq 0$) 时直接取 $\alpha_{\max}$ 。BB 步长的思想是：把相邻两点的差分视为对 Hessian 的割线近似， $\alpha^{(k)}$ 恰为该近似下 H^{-1} 的尺度，从而以极小代价获得近似二阶方法的长步长——这对“不同区域曲率差异悬殊”的非光滑问题光滑化后尤其有效，是算法效率的关键。

沿弧的步长 $\lambda^{(k)}$ 由非单调 Armijo 条件确定，即取 $\lambda = 1, \tau, \tau^2, \dots$ 中第一个满足

$$F_\mu(u^{(k)} + \lambda d^{(k)}) \leq \max_{0 \leq j \leq \min\{M, k\}} F_\mu(u^{(k-j)}) + \delta \lambda (g^{(k)})^\top d^{(k)} \quad (14)$$

者，其中 $d^{(k)} = w^{(k)} - u^{(k)}$ ， $\delta = 10^{-4}$ 为下降系数， $\tau = 0.5$ 为回溯因子， $M = 10$ 为非单调记忆长度。允许目标值在 M 步窗口内暂时上升，可显著改善 BB 步长波动时的接受率。算法流程见算法 1。

算法 1 两阶段椒盐噪声恢复 (GPSR-BB 投影梯度)

输入：噪声图像 Y 、噪声等级 r 、参数 α, β, μ 、终止容差 tolP

输出：恢复图像 u^*

1: 第一阶段：按 r 查表得 $w_{\max}$ ，执行 AMF 得候选集 Ω 与初值 $u^0 = \hat{y}|_\Omega$

2: 第二阶段：按式 (10) 构造盒约束光滑模型 F_μ ， $g^0 = \nabla F_\mu(u^0)$ ， $\alpha^{(0)} = \min\{\alpha_{\max}, \max\{\alpha_{\min}, 1/\|g^0\|_\infty\}\}$

3: **for** $k = 0, 1, 2, \dots$ **do**

$w^{(k)} = P(u^{(k)} - \alpha^{(k)} g^{(k)})$; $d^{(k)} = w^{(k)} - u^{(k)}$

求满足式 (14) 的 $\lambda^{(k)}$; $u^{(k+1)} = u^{(k)} + \lambda^{(k)} d^{(k)}$

$g^{(k+1)} = \nabla F_\mu(u^{(k+1)})$; 按式 (13) 更新 $\alpha^{(k+1)}$

if $\|P(u^{(k+1)} - g^{(k+1)}) - u^{(k+1)}\|_\infty \leq \text{tolP}$ **then** 返回 $u^* = u^{(k+1)}$

end for

5.7 参数选择

正则权重 β 与势函数参数 α 对恢复质量有直接影响。由于 $\varphi_\alpha = \sqrt{t^2 + \alpha}$ 关于 α 存在较宽平台，本文在 Lena 图上对 β 与 α 做网格扫描（种子 2026）。结果表明：PSNR 随 β 增大后在 $\beta \geq 40$ 时进入平台（ $\beta = 40$ 时 38.357 dB，提高到 640 仅再提高 0.008 dB）； α 在 300 附近最优，过小则曲率过强、过大则正则过弱。因此主配置取 $\alpha = 300$ ， $\beta = 40$ 。

5.8 评价指标

为评价恢复质量与算法效率，构建如下指标（均在未取整的浮点恢复图 $\hat{x}$ 上计算，其中 $\hat{x}|_{\Omega} = u^*$ 、 $\hat{x}|_{\Omega^c} = y$ ）：

$$\text{PSNR} = 10 \log_{10} \frac{(s_{\max} - s_{\min})^2}{\text{MSE}}, \quad \text{SNR} = 10 \log_{10} \frac{\sum_{i,j} x_{i,j}^2}{\sum_{i,j} (x_{i,j} - \hat{x}_{i,j})^2}, \quad (15)$$

其中 $\text{MSE} = \frac{1}{MN} \sum_{i,j} (x_{i,j} - \hat{x}_{i,j})^2$ 为均方误差；对本文 8 位灰度图， $s_{\max} - s_{\min} = 255$ 。PSNR 从峰值信号角度度量失真，SNR 从信号能量角度度量噪声水平。另记录平均绝对误差 $\text{MAE} = \frac{1}{MN} \sum |x - \hat{x}|$ 、检测阶段的真噪声检出率 TPR（正确检出的真噪声像素占全部真噪声像素的比例）与误检率 FPR（误判为噪声的干净像素占全部干净像素的比例），以及算法迭代数与求解时间。

5.9 求解结果与分析

本文选取 12 张 512×512 灰度测试图，在 30% 和 50% 两种噪声等级下分别进行 3 次独立实验，共得到 72 组算例。模型参数取 $\alpha = 300$ ， $\beta = 40$ ， $\mu = 1$ ，求解器参数为 $\delta = 10^{-4}$ 、 $\tau = 0.5$ 、 $\text{tolP} = 10^{-2}$ 、迭代上限 1500，全部算例均满足终止准则。实验结果见表 2–表 4，Lena 图像的恢复效果见图 4。

表 2 问题一恢复结果汇总

噪声等级	迭代数	时间 s	PSNR dB	SNR dB	误检率
30%	86.8 [43, 204]	2.94 [1.52, 7.08]	37.33 [30.99, 48.17]	31.88	0.0%–0.6%
50%	92.4 [49, 203]	5.42 [2.93, 12.3]	33.76 [28.29, 43.36]	28.31	0.0%–0.7%

表 3 12 张测试图的恢复结果（3 组随机种子均值）

图像	30% PSNR	30% SNR	30% 迭代	50% PSNR	50% SNR	50% 迭代
cameraman	39.99	34.37	73	35.40	29.79	81
house	48.17	43.46	44	43.36	38.65	53
jetplane	38.20	35.36	82	34.09	31.26	90
lake	34.68	29.51	173	31.10	25.93	154
lena	38.35	32.70	86	34.71	29.05	94
livingroom	34.86	28.93	88	31.17	25.23	86
mandril	33.75	28.20	61	29.72	24.16	61
peppers	37.00	31.06	86	34.18	28.24	92
pirate	35.54	29.10	105	32.16	25.71	100
walkbridge	31.48	25.39	100	28.29	22.20	98
woman_blonde	30.99	25.88	86	29.21	24.10	136
woman_darkhair	44.90	38.65	57	41.67	35.42	65

表 4 不同恢复方法的 PSNR 对比（12 图均值，单位 dB）

噪声等级	噪声图	中值滤波 3×3	中值滤波 7×7	AMF 直接输出	本文方法
30%	10.51	23.35	26.92	32.51	37.33
50%	8.28	15.10	25.66	29.13	33.76



图 4 Lena 图在 30% 椒盐噪声下的恢复结果及局部放大

由表 2 可知，算法在 30% 和 50% 噪声下的平均 PSNR 分别为 37.33 dB 和 33.76 dB，真噪声检出率均不低于 99.8%，误检率不超过 0.7%。其中，Lena 图像在 30% 噪声下的平均 PSNR 为 38.35 dB。由图 4 可见，恢复图像中的脉冲噪声基本消除，图像边缘和局部纹理得到较好保留。

由表 4 可知，与固定窗口中值滤波相比，本文方法在两种噪声等级下均取得更高的 PSNR；在 AMF 直接输出的基础上，平均 PSNR 进一步提高约 4.8–5.1 dB。算法平均迭代约 87 次和 92 次，求解时间随候选像素数量增加而上升，与复杂度分析一致。

此外，为检验建模中关键选择的必要性，在 Lena 与 Peppers 图上对求解器、光滑参数与数据保真项做对照实验，结果见表 5。可得三点结论：其一，BB 步长是求解效率的关键——GPSR-Basic 需 7160 步 266 s，GPSR-BB 仅 85 步 3.19 s，迭代数与时间在三幅对照图上分别降低约 60–88 倍与 50–83 倍，恢复质量完全相同。其二，光滑参数 μ 在本配置下由逼近误差决定而非求解效率——消融实验（存档见支撑材料）显示 μ 从 1 降到 10^{-4} 时迭代数均为 85 步、恢复 PSNR 差不超过 0.0001 dB， $\mu = 1$ 是精度与工程便利的双赢选择。其三，数据保真项在 β 足够大时可以删除——无数据项版本 PSNR 为 38.366 dB，比含数据项高 0.009 dB，验证了“第二阶段数据项可删”的论断。

表 5 问题一求解策略对比（Lena 图， $r = 30\%$ ）

策略	迭代数	时间 s	PSNR dB
GPSR-Basic 沿投影弧回溯	7160	265.86	38.357
GPSR-BB 非单调（主配置）	85	3.19	38.357
GPSR-BB BB2 步长	79	2.96	38.357
GPSR-BB μ 续延 $(1, 10^{-1}, 10^{-2}, 10^{-3})$	88	3.28	38.357
无数据保真项版本	97	3.69	38.366

六、 问题二模型的建立与求解

6.1 五种保边势函数及其数学性质

问题二沿用问题一的两阶段框架，保持噪声检测、数据保真项、邻域结构和求解算法不变，仅替换正则项中的保边势函数。对第 k 种势函数，将模型中的 φ_α 替换为 $\varphi_\alpha^{(k)}$ ，其余条件保持一致。五种候选势函数为 [3]

$$\begin{aligned}
 \varphi_\alpha^{(1)}(t) &= \sqrt{t^2 + \alpha}, \quad \alpha > 0, & \varphi_\alpha^{(2)}(t) &= |t|^\alpha, \quad 1 < \alpha \leq 2, \\
 \varphi_\alpha^{(3)}(t) &= \log \cosh(\alpha t), & \varphi_\alpha^{(4)}(t) &= \frac{|t|}{\alpha} - \log \left(1 + \frac{|t|}{\alpha} \right), \\
 \varphi_\alpha^{(5)}(t) &= \begin{cases} \frac{t^2}{2\alpha}, & |t| \leq \alpha, \\ |t| - \frac{\alpha}{2}, & |t| > \alpha. \end{cases}
 \end{aligned}$$

五种势函数均为偶凸函数，并随 $|t|$ 单调增加，其中 t 表示相邻像素的灰度差。当 $|t|$ 较小时，势函数通过惩罚局部波动抑制噪声；当 $|t|$ 较大时，惩罚增长速度降低，从而减少对真实边缘的平滑。不同势函数的主要差异体现在原点附近的曲率、远离原点后的增长速度以及导数是否有界，具体性质见表 6，函数形状见图 5。

表 6 五种势函数的解析性质与光滑化方式

势函数	光滑类	导数界 $ \varphi' \leq$	原点二阶导	光滑化方式
平方根型	C^∞	1	$1/\sqrt{\alpha}$	直接使用
幂函数型	C^1	无界	奇异	$(t^2 + \mu^2)^{\alpha/2} - \mu^\alpha$
对数余弦型	C^∞	α	α^2	直接使用
对数型	C^2	$1/\alpha$	$1/\alpha^2$	直接使用
Huber 型	C^1	1	$1/\alpha$	直接使用

表 6 中的光滑性和导数界直接影响目标函数的曲率，进而影响梯度算法的稳定性与收敛速度。因此，势函数的比较既要考察恢复质量，也要记录迭代次数。

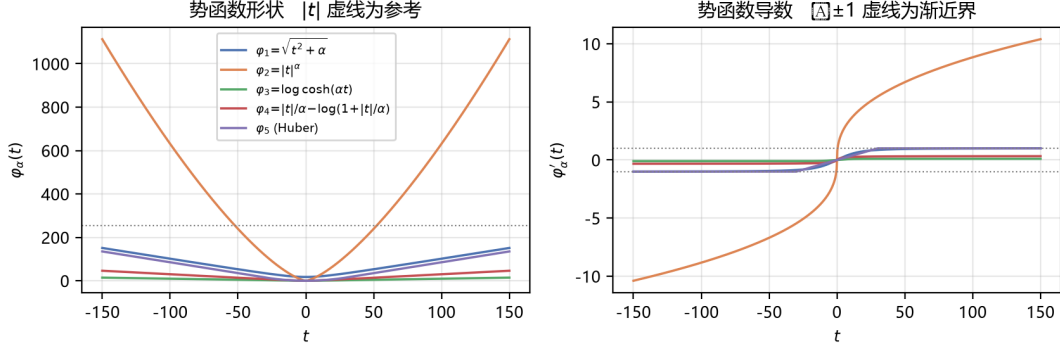


图 5 五种保边势函数及其导数曲线

6.2 幂函数型的光滑化处理

当 $1 < \alpha < 2$ 时，幂函数 $|t|^\alpha$ 在 $t = 0$ 附近的二阶导数无界，可能导致梯度变化过快，降低迭代算法的稳定性。为此，本文采用光滑化处理，将 $\varphi_\alpha^{(2)}$ 近似为

$$\varphi_{\alpha,\mu}^{(2)}(t) = (t^2 + \mu^2)^{\alpha/2} - \mu^\alpha, \quad (16)$$

当 $|t| \gg \mu$ 时，该函数与原幂函数近似一致；在 $t = 0$ 附近则保持光滑，从而便于使用梯度算法求解。减去与 t 无关的常数 μ^α 可保持 $\varphi_{\alpha,\mu}^{(2)}(0) = 0$ ；该常数不改变梯度与极小点，但使不同 μ 下的函数值具有一致基准。

6.3 公平比较协议

五种势函数中参数 α 的含义不同，若直接使用同一组参数，结果可能主要反映参数尺度差异，而不能公平反映势函数本身的性能。因此，本文分别标定各势函数的参数：先固定 $\beta = 40$ 搜索 α ，取该网格内 PSNR 最大的参数记为 $\hat{\alpha}$ ；再固定 $\alpha = \hat{\alpha}$ ，搜索 $\beta \in \{5, 20, 40, 80, 160, 320, 640\}$ ，在满足停止准则的结果中取该网格内 PSNR 最大者记为 $\hat{\beta}$ 。先确定势函数形状，再确定正则项权重，可以减少参数组合数量，并使各函数在各自较合适的尺度下比较。该过程是一次坐标式标定，不能保证得到 α 与 β 的联合全局最优值；因此本文结论限定在给定网格与标定协议内。

参数标定使用 Lena 图像及随机种子 2026；参数迁移验证使用其余图像与随机种子，并在汇总时同时列入 Lena 的标定实例。正式汇总覆盖 Lena、Cameraman 和 Peppers 三幅图像、两个噪声等级及两组随机种子（2026、7）。因此该实验包含样本外的参数迁移检验，但汇总均值不是完全独立的样本外测试。各方法采用相同的噪声图、停止准则和最大迭代次数，并使用 PSNR、SSIM 和边缘 PSNR 评价恢复质量，使用迭代次数评价求解效率。

6.4 参数敏感性分析

各势函数的 PSNR 随参数 α 的变化曲线见图 6。

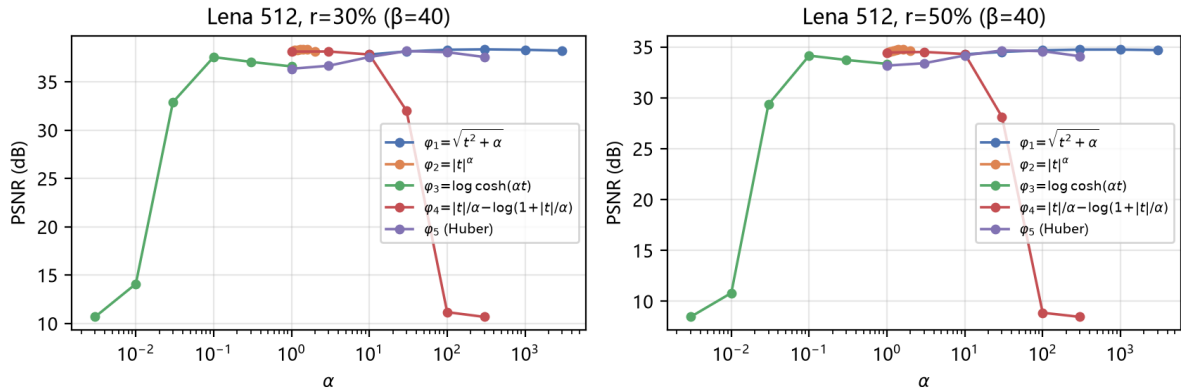


图 6 五种势函数的参数 α 敏感性

由图 6 可知，平方根型和 Huber 型在较宽参数区间内保持稳定；对数余弦型和对数型对参数较为敏感，参数偏离有效区间时 PSNR 明显下降；幂函数经过光滑化后，其恢复结果随参数变化趋于稳定。因此，参数稳定性也是选择势函数时需要考虑的因素。

6.5 对比实验结果与结论

按标定协议得到各势函数的网格内选定参数后做正式对比，结果见表 7。

表 7 五种势函数的恢复结果比较

r	势函数	$\hat{\alpha}$	$\hat{\beta}$	PSNR dB	SSIM	边缘 PSNR	迭代数
30%	平方根型	300	640	38.43	0.969	34.59	116
30%	幂函数型	1.4	640	38.39	0.969	34.56	137
30%	对数余弦型	0.1	640	38.29	0.969	34.44	608
30%	对数型	3	640	38.37	0.968	34.56	521
30%	Huber 型	30	640	38.24	0.969	34.36	757
50%	平方根型	1000	640	34.70	0.941	30.62	53
50%	幂函数型	1.6	160	34.73	0.941	30.65	85
50%	对数余弦型	0.1	640	34.64	0.941	30.57	608
50%	对数型	3	640	34.68	0.940	30.62	611
50%	Huber 型	30	640	34.60	0.941	30.50	740

由表 7 可知，五种势函数在相同噪声等级下的恢复质量接近。30% 和 50% 噪声下的 PSNR 极差分别为 0.19 dB 和 0.13 dB，SSIM 及边缘 PSNR 差异较小，说明在参数适当时，五种势函数均能实现较好的保边恢复。正则权重的敏感性见图 7；多数曲线在 β 增大后进入平台或增益逐步收窄，但不同势函数进入平台的区间并不相同，不能把 $\beta \geq 40$ 视为对所有势函数都成立的统一阈值。

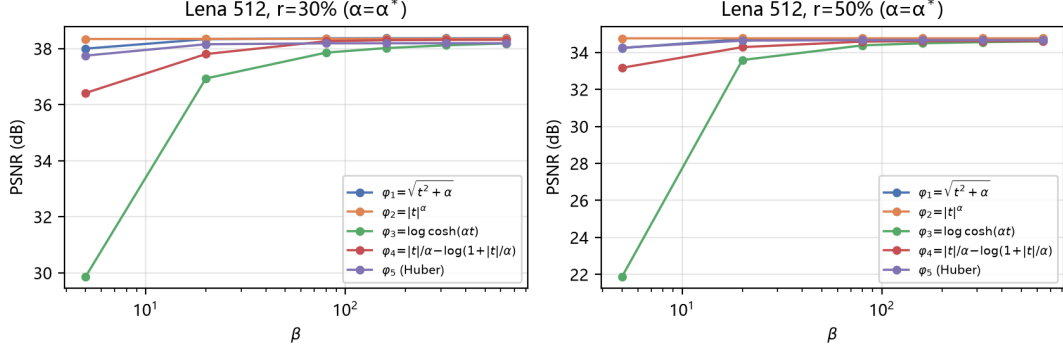


图 7 五种势函数的正则权重 β 敏感性

不同势函数的求解效率差异明显：平方根型和幂函数型约需 50–140 次迭代，对数型、对数余弦型和 Huber 型约需 500–760 次迭代。综合恢复质量、参数稳定性和求解效率，本文优先推荐平方根型或经过光滑化处理的幂函数型势函数。

七、 问题三模型的建立与求解

7.1 稀疏重构模型与实验实例

稀疏信号重构是压缩感知的核心问题：信号 $x \in \mathbb{R}^n$ 仅含 $s \ll n$ 个非零元，通过 $m < n$ 个线性观测 $b = Ax + e$ 恢复 x ，其中 $A \in \mathbb{R}^{m \times n}$ 为测量矩阵， e 为观测噪声。由于方程数少于未知数，欠定系统 $b = Ax$ 有无穷多组解；稀疏性是挑选真解的关键先验。直接以“非零元个数最少”为目标即 ℓ_0 最小化，但该问题是 NP 难的；当 A 满足适当的几何条件（如约束等距性）时，其凸松弛—— ℓ_1 正则化最小二乘

$$\min_x \tau \|x\|_1 + \frac{1}{2} \|Ax - b\|_2^2 \quad (17)$$

——是常用的可计算凸替代；当测量矩阵满足适当条件时，它可实现精确或稳定的稀疏恢复，其中 $\tau > 0$ 为平衡数据拟合与稀疏性的正则参数。

实验实例设置为 [1]： $n = 4096$ ， $m = 1024$ ，稀疏度 $s = 160$ ，非零元位置随机且取值为 ± 1 ； A 为行正交化的标准高斯矩阵；观测 $b = Ax + e$ ，噪声方差 $\sigma^2 = 10^{-4}$ ；正则参数取 $\tau = 0.1 \|A^\top b\|_\infty$ 。共生成 10 组独立实例（种子 0–9）。

7.2 光滑化与修正 PRP 共轭梯度法

7.2.1 共轭梯度法

共轭梯度法（Conjugate Gradient, CG）是求解大规模优化问题的经典迭代法，只需存储若干与变量同维的向量，内存开销 $O(n)$ 。对二次目标函数，共轭梯度法构造的搜索方向两两关于 Hessian 共轭，理论上不超过 n 步即得精确解。对一般非线性目标，

Polak–Ribière–Polyak (PRP) 等共轭参数把这一思想推广为非线性共轭梯度法：

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = \begin{cases} -\nabla F_\mu(x_0), & k = 0, \\ -\nabla F_\mu(x_k) + \beta_k d_{k-1}, & k \geq 1, \end{cases} \quad (18)$$

其中 $g_k = \nabla F_\mu(x_k)$, α_k 由线搜索确定, 共轭参数 β_k 的常见取法有

$$\beta_k^{\text{FR}} = \frac{\|g_k\|^2}{\|g_{k-1}\|^2}, \quad \beta_k^{\text{PRP}} = \frac{g_k^\top y_{k-1}}{\|g_{k-1}\|^2}, \quad \beta_k^{\text{HS}} = \frac{g_k^\top y_{k-1}}{d_{k-1}^\top y_{k-1}}, \quad \beta_k^{\text{DY}} = \frac{\|g_k\|^2}{d_{k-1}^\top y_{k-1}}, \quad (19)$$

其中 $y_{k-1} = g_k - g_{k-1}$ 为相邻梯度差。PRP 参数在精确线搜索下具有“自动重启”的优良数值性质（某步线搜索充分精确时方向退化为负梯度，相当于重新出发），但一般线搜索下其全局收敛性无法保证。修正 PRP (PRP+) 以非负截断修正共轭参数：

$$\beta_k^{\text{PRP+}} = \max \left\{ 0, \frac{g_k^\top y_{k-1}}{\|g_{k-1}\|^2} \right\}. \quad (20)$$

截断保证共轭参数恒非负，配合实现中的充分下降性检查与负梯度重启，可避免方向退化时产生短步甚至停滞，既保留 PRP 数值性能好的特点，又改善异常方向下的稳健性。相应的全局收敛性需要额外条件：在水平集有界、梯度 Lipschitz 连续、线搜索满足强 Wolfe 条件，且 $\beta_k \geq 0$ 并满足 Gilbert–Nocedal 性质 (*) 时， $\liminf_k \|g_k\| = 0$ 。本文实现的是两项 PRP+ 加下降性重启，相关收敛结论只用于说明算法成立所需的条件，并不构成对全部工程分支的直接证明。

线搜索采用强 Wolfe 条件： α_k 需同时满足

$$F_\mu(x_k + \alpha_k d_k) \leq F_\mu(x_k) + c_1 \alpha_k g_k^\top d_k, \quad \left| \nabla F_\mu(x_k + \alpha_k d_k)^\top d_k \right| \leq c_2 \left| g_k^\top d_k \right|, \quad (21)$$

其中 $c_1 = 10^{-4}$ 、 $c_2 = 0.1$ 。第一式给出充分下降条件，第二式限制新点处方向导数的绝对值；二者共同排除过短的无效步长和破坏充分下降的过长步长。实现中另加充分下降保障：若某步方向的下降性不足（ $g_k^\top d_k \geq -10^{-7} \|g_k\|^2$ ）则重置为负梯度方向；强 Wolfe 搜索失败时以负梯度重启与 Armijo 回溯作工程兜底，兜底次数进入诊断计数。需要说明的是，上述收敛结论只对应强 Wolfe 条件实际成立的迭代，本文不将工程实现笼统表述为无条件全局收敛。

7.2.2 算法流程

对模型 (17) 的光滑化形式 $F_\mu(x) = \tau \sum_i \rho_\mu(x_i) + \frac{1}{2} \|Ax - b\|_2^2$ ，其梯度为 $\tau \rho'_\mu(x) + A^\top(Ax - b)$ 。为控制光滑化偏差， μ 按 $10^{-1}, 10^{-2}, \dots, 10^{-6}$ 逐级续延，每段以上一段的解热启动；初值取 $x_0 = A^\top b[4]$ 。每段以相邻两步目标相对变化不超过 10^{-6} 为主要停止准则，该准则在极小 μ 段比梯度范数准则更稳健；同时保留梯度无穷范数不超过 10^{-6} 的提前停止，每段迭代上限 2000。流程见算法 2。

算法 2 光滑化修正 PRP 共轭梯度法 (μ 续延)

输入: 测量矩阵 A 、观测 b 、参数 τ 、 μ 序列 ($10^{-1}, \dots, 10^{-6}$)、相对变化容差 rel_tol

输出: 重构信号 x^*

```

1:  $x \leftarrow A^\top b$ 
2: for 每一级  $\mu$  (由大到小) do
     $g \leftarrow \nabla F_\mu(x)$ ;  $d \leftarrow -g$ 
    repeat
        强 Wolfe 线搜索求  $\alpha$ :  $x_{\text{new}} \leftarrow x + \alpha d$ ;  $g_{\text{new}} \leftarrow \nabla F_\mu(x_{\text{new}})$ 
        计算目标相对变化
        if 目标相对变化  $\leq \text{rel\_tol}$  then  $x \leftarrow x_{\text{new}}$ ;  $g \leftarrow g_{\text{new}}$ ; 结束本段
        按式 (20) 计算  $\beta$ :  $d \leftarrow -g_{\text{new}} + \beta d$ 
        if 下降性不足 then  $d \leftarrow -g_{\text{new}}$ 
         $x \leftarrow x_{\text{new}}$ ;  $g \leftarrow g_{\text{new}}$ 
    until 本段终止
end for; 返回  $x^* = x$ 

```

7.3 GPSR-BCQP 对比算法

GPSR 方法 [1] 采用变量分裂: 把 x 的正部与负部分开, 令 $x = u - v$ 、 $u, v \geq 0$, 合并为 $z = [u; v] \geq 0$, 则 $\|x\|_1 = \mathbf{1}^\top z$, 模型 (17) 化为有界约束二次规划 (BCQP)

$$\min_{z \geq 0} \frac{1}{2} z^\top B z + c^\top z, \quad B = \begin{bmatrix} A^\top A & -A^\top A \\ -A^\top A & A^\top A \end{bmatrix}, \quad c = \tau \mathbf{1} + \begin{bmatrix} -A^\top b \\ A^\top b \end{bmatrix}, \quad (22)$$

其中 B 为半正定二次型矩阵, c 为线性项。对该二次目标, 投影梯度迭代 $w = (z - \alpha \nabla F)_+$ 、 $z \leftarrow z + \lambda(w - z)$ 的弧搜索步长有闭式解 $\lambda = \text{mid}(0, -g^\top s / (s^\top B s), 1)$ (二次函数沿弧可精确最小化), 步长 α 取 BB secant 更新, 终止准则取 $\|\min(z, \nabla F(z))\|_2 \leq 10^{-3}$ 。本文实现 GPSR-BB 工程版本, 并采用去偏置过程 [1]: 固定解的支撑集后做最小二乘精化, 即仅对 $|\hat{x}_i|$ 显著非零的分量用 $\min \|A_S x_S - b\|_2$ 重解, 以消除 ℓ_1 正则对幅度的系统性收缩。

7.4 求解结果与分析

10 组独立实例上共对 9 个配置做对比, 包括 5 种共轭参数、带续延与不延续的修正 PRP 两种配置、以及 GPSR-BB 与去偏置版本。结果均值见表 8, 收敛过程与信号恢复对比见图 8。

第一, 支撑恢复能力。全部方法都恢复出全部 160 个真尖峰; 未去偏置时相对误差约 0.24, 这是 ℓ_1 正则对幅度的系统收缩所致; 去偏置后误差下降约一个数量级: 修正 PRP 续延加去偏置与 GPSR-BB 加去偏置的相对误差分别为 0.0286 与 0.0265, 支撑误报数 3.9 对 2.8, 恢复质量基本持平。这得益于续延把光滑化偏差从 $\mu = 10^{-3}$ 时的 0.50% 压到 $\mu = 10^{-6}$ 时约 0.01%。

表 8 问题三主实验结果均值， fp 为支撑误报数，真尖峰数 **160**，续延修正 **PRP** 段停止采用相对目标变化准则；**GPSR-BB** 去偏置行的迭代数含固定支撑最小二乘精化 **1** 步

方法	迭代数	时间 s	相对误差	fp
GPSR-BB	29.1	0.14	0.2366	8.7
GPSR-BB 去偏置	30.1	0.14	0.0265	2.8
修正 PRP 单段	168	2.16	0.2444	10.9
原始 PRP 无截断	168	2.13	0.2444	10.9
FR 共轭参数	1076	20.0	0.2428	9.8
HS 共轭参数	173	2.15	0.2437	10.3
DY 共轭参数	1084	20.4	0.2428	9.9
修正 PRP 续延	180	2.77	0.2420	10.2
修正 PRP 续延去偏置	180	2.77	0.0286	3.9

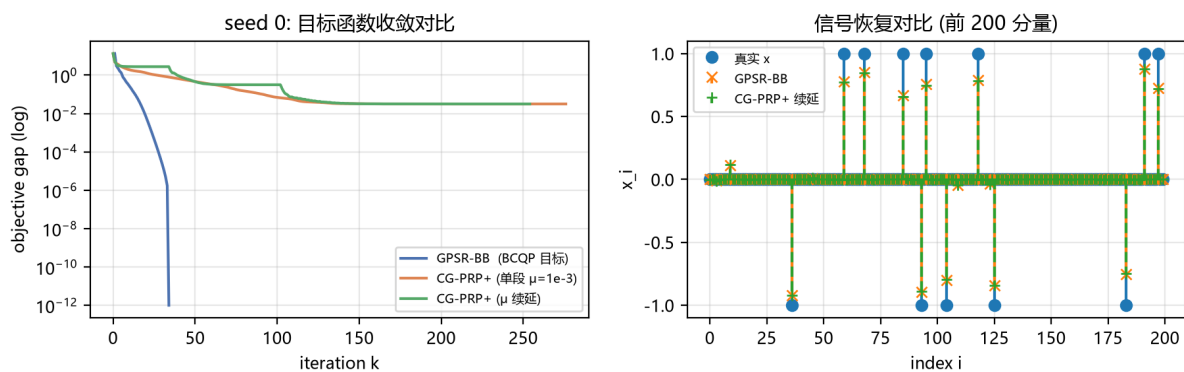


图 8 单实例的目标函数收敛对比与信号恢复对比，左图纵轴为对数尺度

第二，收敛速度。GPSR-BB 以 29 步、0.14 s 收敛，修正 PRP 续延需 180 步、2.77 s。按“达到相同目标值”口径，GPSR-BB 需 29.1 步 0.14 s，修正 PRP 需 122.7 步 1.64 s，为前者的 4.2 倍。初值与停止准则对共轭梯度法的影响显著：取零初值并把梯度范数准则用于各段时，同目标值口径需 257.3 步；初值改为 $A^T b$ 并按相对变化准则分段停止后降为 122.7 步，说明初值选择捕获了观测的主要能量方向，而相对变化准则避免了极小 μ 段的梯度病态。共轭梯度法的价值体现在存储量仅 $O(n)$ 、无需变量分裂与投影，可直接处理问题一、二那样不存在显式二次型的光滑目标，其速度劣势是“二次结构 + 闭式弧搜索”下的算法匹配性问题。

第三，共轭参数的选择。FR 与 DY 参数需约 1080 步，是修正 PRP 与 HS 参数迭代数的 6 倍以上，验证了修正 PRP 与 HS 在本组实例上的数值优势。原始 PRP 与修正 PRP 使用相同初值、停止条件、线搜索和下降性保障，仅截断规则不同；两者结果一致，且诊断记录中负 β 和截断次数均为零，说明截断在本组实例上未被触发，当前实验不能单独证明其数值增益。

八、 灵敏度分析与模型检验

8.1 光滑参数的灵敏度

问题一的消融实验（表 5 与支撑材料存档）显示 μ 从 1 降到 10^{-4} 时迭代数与恢复 PSNR 均几乎不变，说明该配置下恢复质量与收敛效率对 μ 均不敏感， $\mu = 1$ 的选择由逼近误差决定。问题三的灵敏度方向不同： μ 控制光滑化偏差， $\mu = 10^{-3}$ 时 ℓ_1 目标值偏差 0.50%， $\mu = 10^{-6}$ 时降为 0.01%，而 $\mu = 10^{-5}$ 以后线搜索开始退化，说明最优精度存在下界。综合两题， μ 的选取应与其控制对象匹配：图像恢复中逼近误差允许达到一个灰度级，稀疏重构中则需要更精细的控制。

8.2 正则权重与势函数参数的灵敏度

问题一的 β 网格扫描显示 $\beta \geq 40$ 后进入平台。问题二中 β 扫描上界扩展后，对数型与对数余弦型的选定权重从 160 移至 640，PSNR 分别提升 0.03 dB 与 0.12 dB，说明网格截断会系统性地低估这类势函数的性能。为检验 $\hat{\beta}$ 落在网格上界 640 是否造成低估，本文在各势函数标定 $\hat{\alpha}$ 处补充 $\beta = 1280$ 的延伸验证：平方根型、幂函数型、对数型与 Huber 型的 PSNR 增益均不超过 0.001 dB，平台结论成立；对数余弦型再提高 0.046 dB 与 0.022 dB，增量较前一级（+0.066 dB 与 +0.036 dB）继续收窄、趋于渐近，即便计入该增益其排序仍不变，验证数据见支撑材料。

8.3 随机性与实现的鲁棒性

同一图像同一噪声等级下，三组随机种子（2026、7、42）的 PSNR 标准差为 0.005–0.42 dB，说明算法对噪声的随机实现不敏感。实现层面还需说明两点：其一，求解器对 BB 步长中曲率信息失效的情况取 $\alpha_{\max}$ ，并对投影弧方向做下降性检查，避免数值保护分支造成停滞；其二，梯度与目标函数的求值经过有限差分校验，问题一三种光滑函数与问题二五种势函数（20 样本）的最大相对误差均不超过 6×10^{-5} ，计算正确性可靠。

九、 模型的评价与推广

9.1 模型的优点

第一，模型链条完整且具有可解释性。本文从噪声模型出发，经两阶段检测、边和形式的泛函改写、光滑化与盒约束化，逐步建立起可求解的凸规划，每一步都有明确的物理或数学依据；梯度计算仅依赖四邻域查值，复杂度可控且便于实现。

第二，模型设计中的关键步骤均给出明确的数学定义。检测阶段采用两条件候选集，把误检率压缩到 1% 以内；势函数采用光滑函数族与 ℓ_p 型光滑处理，停止准则和初值选择也给出具体实现口径，便于复核模型与计算结果。

第三，实验设计遵循严谨的对比协议。问题一设置了基线方法、求解器、光滑参数与数据项多个维度的对照，问题二采用独立标定并补充剔除标定图像的样本外汇总，问题三引入“达到相同目标值”的统一口径，主要评价指标均在多随机种子下汇总，便于复核结果的稳定性。

9.2 模型的缺点

第一，光滑参数 μ 与正则权重 β 的取值依赖实验标定。问题一通过扫描确定 $\mu = 1$ 与 $\beta \geq 40$ 的合适区间，但缺乏自适应的在线选择准则，对不同类型的图像或噪声水平可能需要重新标定。

第二，共轭梯度法在小 μ 段需要靠相对目标变化准则避免梯度病态，该准则的容差本身需要经验选取，且问题三中其绝对速度明显逊色于 GPSR-BB，反映其适用场景受限。

第三，本文的对比仅覆盖灰度标准图像与单一噪声模型，对彩色图像、随机值噪声与混合噪声的推广效果尚未验证，结论的外推需谨慎。

9.3 模型的推广

第一，两阶段框架可直接推广到随机值噪声与混合噪声场景，只需调整检测阶段的判决规则，候选集的两条件定义可替换为对应噪声模型的取值特征。

第二，问题二建立的势函数对比协议可推广到其他正则项形式，如总变分、非凸保边函数，只需补充相应导数与凸性分析。

第三，问题三的光滑化续延策略可用于非负稀疏重构、压缩感知中的其他 ℓ_p 正则问题，此时只需替换光滑函数族，并沿用相对目标变化的分段停止准则。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于问题分析与建模思路讨论、代码编写与调试、实验数据核算与文稿语言润色，详细使用情况见支撑材料。

参考文献

- [1] Figueiredo M A T, Nowak R D, Wright S J. Gradient Projection for Sparse Reconstruction: Application to Compressed Sensing and Other Inverse Problems[J]. IEEE Journal of Selected Topics in Signal Processing, 2007, 1(4): 586-597. DOI: 10.1109/JSTSP.2007.910281.
- [2] Chan R H, Ho C W, Nikolova M. Salt-and-pepper noise removal by median-type noise detectors and detail-preserving regularization[J]. IEEE Transactions on Image Processing,

- 2005, 14(10): 1479-1485.
- [3] Cai J F, Chan R H, Fiore C D. Minimization of a Detail-Preserving Regularization Functional for Impulse Noise Removal[J]. Journal of Mathematical Imaging and Vision, 2007, 29(1): 79-91.
 - [4] Wu C, Wang J, Alcantara J H, et al. Smoothing Strategy Along with Conjugate Gradient Algorithm for Signal Reconstruction[J]. Journal of Scientific Computing, 2021, 87(21): 1-18. DOI: 10.1007/s10915-021-01440-z.
 - [5] 刘莹, 朱志斌, 丁玥宏, 等. Wolfe 线搜索下一个新的共轭梯度法及其在信号处理中的应用 [J]. 应用数学, 2025, 38(1): 104-113.
 - [6] 古恒洋, 杜学武. 一类带有重启步的混合截断谱共轭梯度法及其在图像恢复中的应用 [J/OL]. 系统科学与数学, 2024. DOI: 10.12341/jssms240367.
 - [7] 杨艳雪, 杜守强, 吕施春. 一种新的求解含有 ℓ_1 范数优化问题的共轭梯度法 [J]. 应用数学学报, 2024, 47(4): 643-655.
 - [8] Narushima Y, Yabe H, Ford J A. A Three-Term Conjugate Gradient Method with Sufficient Descent Property for Unconstrained Optimization[J]. SIAM Journal on Optimization, 2011, 21(1): 212-230.
 - [9] Chen X, Zhou W. Smoothing nonlinear conjugate gradient method for image restoration using nonsmooth nonconvex minimization[J]. SIAM Journal on Imaging Sciences, 2010, 3(4): 765-790.
 - [10] Gilbert J C, Nocedal J. Global convergence properties of conjugate gradient methods for optimization[J]. SIAM Journal on Optimization, 1992, 2(1): 21-42.

附录一 支撑材料清单

本文全部程序与结果数据在项目仓库 `signal-reconstruction/` 中，代码文件见表 9，主要结果文件见表 10。

表 9 支撑材料：代码文件清单

文件	功能说明
<code>src/noise_model.py</code>	椒盐噪声生成、标准测试图读取
<code>src/amf.py</code>	第一阶段自适应中值滤波检测（算法 1 第 1 步）
<code>src/restoration_model.py</code>	第二阶段光滑化保边泛函：目标、梯度、5 种势函数
<code>src/solvers.py</code>	GPSR-Basic / GPSR-BB 投影梯度求解器（算法 1）
<code>src/sparse_reconstruction.py</code>	问题三实例生成、光滑化 ℓ_1 二次模型
<code>src/cg_solvers.py</code>	非线性共轭梯度（算法 2）与 GPSR-BCQP 对比算法
<code>src/metrics.py</code>	PSNR / SNR / MAE / SSIM / 边缘 PSNR / 检测统计
<code>exp/problem1/run_problem1.py</code>	问题一主实验（12 图 $\times$ 2 等级 $\times$ 3 种子）
<code>exp/problem1/strategy_compare.py</code>	问题一策略对比与 μ 消融
<code>exp/problem2/tune_potentials.py</code>	问题二 (α, β) 独立标定
<code>exp/problem2/verify_beta1280.py</code>	问题二 $\beta = 1280$ 网格上界验证
<code>exp/problem2/run_problem2.py</code>	问题二主实验与样本外汇总
<code>exp/problem3/run_problem3.py</code>	问题三 10 实例 9 配置对比实验
<code>exp/problem3/init_stop_ablation.py</code>	问题三初值与停止准则消融
<code>tests/test_problem3_regressions.py</code>	回归测试

表 10 支撑材料：主要结果文件清单

文件	内容说明
<code>exp/problem1/results/tables/problem1_full.csv</code>	问题一 72 算例逐次运行明细
<code>exp/problem1/results/tables/problem1_summary.csv</code>	问题一逐图均值汇总
<code>exp/problem1/results/tables/problem1_strategy.txt</code>	问题一求解器 / μ / 数据项对照
<code>exp/problem1/results/tables/problem1_mu_ablation.csv</code>	问题一光滑参数 μ 消融
<code>exp/problem2/results/tables/problem2_tuning.txt</code>	问题二 (α, β) 标定日志
<code>exp/problem2/results/tables/problem2_summary.csv</code>	问题二主实验汇总
<code>exp/problem2/results/tables/problem2_summary_oos.csv</code>	问题二样本外（剔除 Lena）汇总
<code>exp/problem2/results/tables/problem2_beta1280_verification.csv</code>	$\beta = 1280$ 延伸验证
<code>exp/problem3/results/tables/problem3_full.csv</code>	问题三 10 实例逐次明细
<code>exp/problem3/results/tables/problem3_summary.csv</code>	问题三 9 配置均值汇总
<code>exp/problem3/results/tables/problem3_init_stop_ablation.csv</code>	初值与停止准则消融

附录二 主要程序代码

以下给出四个核心程序（Python 3.11，依赖 NumPy / SciPy），完整代码见支撑材料。

程序 1 自适应中值滤波检测 (src/amf.py)

```
1 def adaptive_median_filter(y, r=0.3, wmax=None, s_min=0.0, s_max=255.0):
2     """AMF 检测: 逐级扩窗, 返回噪声候选集 cand 与 AMF 输出 y_amf."""
3     y = np.asarray(y, dtype=np.float64)
4     M, N = y.shape
5     if wmax is None:
6         wmax = w_max_for_noise_level(r)          # 按噪声等级设定最大窗口
7     smin = np.full((M, N), np.nan)               # 各像素最终窗口统计
8     smed = np.full((M, N), np.nan)
9     smax = np.full((M, N), np.nan)
10    active = np.ones((M, N), dtype=bool)          # 尚未确定最终窗口的像素
11    for w in range(3, wmax + 1, 2):               # 3,5,7,...,w_max 逐级扩窗
12        if not np.any(active):
13            break
14        # 只在活跃像素的包围盒(外扩 w//2)内计算窗口统计, 结果与全图一致
15        idx = np.where(active); pad = w // 2
16        i0 = max(int(idx[0].min()) - pad, 0); i1 = min(int(idx[0].max()) + pad + 1, M)
17        j0 = max(int(idx[1].min()) - pad, 0); j1 = min(int(idx[1].max()) + pad + 1, N)
18        lo, med, hi = _window_stats(y[i0:i1, j0:j1], w, mode="reflect")
19        chk_sub = active[i0:i1, j0:j1] & (lo < med) & (med < hi)
20        chk = np.zeros((M, N), dtype=bool); chk[i0:i1, j0:j1] = chk_sub
21        smin[chk], smed[chk], smax[chk] = lo[chk_sub], med[chk_sub], hi[chk_sub]
22        active &= ~chk                             # 满足 smin<smed<smax 即停止扩窗
23    if np.any(active):                             # 窗口耗尽: 用 wmax 窗口统计
24        lo, med, hi = _window_stats(y, wmax, mode="reflect")
25        smin[active], smed[active], smax[active] = lo[active], med[active], hi[active]
26    exhausted = active.copy()                       # 从未获得满足窗口的像素
27    amf_cand = exhausted | ~((smin < y) & (y < smax)) # AMF 判为噪声的像素
28    y_amf = y.copy(); y_amf[amf_cand] = smed[amf_cand] # 候选像素用中值替换
29    # 两条件候选集: y 为端点值且 AMF 输出了替换值
30    cand = ((y == s_min) | (y == s_max)) & (y_amf != y)
31    return cand, y_amf, dict(smin=smin, smed=smed, smax=smax)
```

程序 2 光滑化保边泛函的求值与梯度 (src/restoration_model.py)

```
1 class Phase2Model:
2     """光滑化第二阶段模型:  $F_{\mu}(u) = \sum \rho(u-y) + \beta \sum_{\text{edge}} \phi(u_p - u_q)$ ."""
3
4     def __init__(self, y, mask, beta=5.0, alpha=100.0,
5                  smooth="sqrt", potential="sqrt", boundary="reflect"):
6         self.y = np.asarray(y, dtype=np.float64)
7         self.mask = np.asarray(mask, dtype=bool)          # 候选集 Omega
8         self.beta, self.alpha = float(beta), float(alpha)
9         self.phi, self.dphi, self.dphi_sm = POTENTIALS[potential]
10        self.rho, self.drho = SMOOTHINGS[smooth]
11        self.mu, self.with_data = 1.0, True
12        self.nvars = int(np.sum(self.mask))
13        self.idx_map = np.full(self.mask.shape, -1, dtype=np.int64)
14        self.idx_map[self.mask] = np.arange(self.nvars, dtype=np.int64)
15        self.coords = np.argwhere(self.mask)
16        self._build_topology()                             # 预计算四邻域索引表
17
18        def value(self, u):                                # 目标函数  $F_{\mu}(u)$ 
19            u = np.asarray(u, dtype=np.float64)
20            yN = self.y[self.mask]
21            fdata = np.sum(self.rho(u - yN, self.mu)) if self.with_data else 0.0
22            freg = 0.0
23            for d in range(self.ndirs):                    # 4 个邻域方向
24                k = self.nb_kind[d]
25                m0 = np.where(k == 0)[0]                  # 邻居为干净像素:  $2 \times \phi$ 
26                if m0.size:
```



```

27         freg += 2.0 * np.sum(self.phi_sm(u[m0] - self.nb_yval[d][m0],
28                                     self.alpha, self.mu))
29         m1 = np.where(k == 1)[0]          # 邻居为候选变量: phi
30         if m1.size:
31             freg += np.sum(self.phi_sm(u[m1] - u[self.nb_idx[d][m1]],
32                                         self.alpha, self.mu))
33         return fdata + 0.5 * self.beta * freg
34
35     def gradient(self, u):                  # 梯度(四邻域查值)
36         u = np.asarray(u, dtype=np.float64)
37         yN = self.y[self.mask]
38         g = self.drho(u - yN, self.mu) if self.with_data else np.zeros_like(u)
39         for d in range(self.ndirs):
40             k = self.nb_kind[d]
41             m0 = np.where(k == 0)[0]
42             if m0.size:
43                 g[m0] += self.beta * self.dphi_sm(u[m0] - self.nb_yval[d][m0],
44                                                     self.alpha, self.mu)
45             m1 = np.where(k == 1)[0]
46             if m1.size:
47                 g[m1] += self.beta * self.dphi_sm(u[m1] - u[self.nb_idx[d][m1]],
48                                                     self.alpha, self.mu)
49         return g

```

程序 3 GPSR-BB 投影梯度求解器 (src/solvers.py)

```

1  def gpsr_bb(model, u0, mu=1e-3, lo=0.0, hi=255.0, tolP=1e-2, tolX=1e-10,
2          maxit=5000, amin=1e-10, amax=1e6, delta=1e-4, tau=0.5,
3          lsearch_max=30, M=10, bb_variant=1):
4      """BB 步长 + 投影 + 非单调 Armijo 线搜索 (算法 1)."""
5      model._set_mu(mu)
6      z = np.asarray(u0, dtype=np.float64).copy()
7      g = model.gradient(z)
8      a0 = 1.0 / max(1.0, float(np.max(np.abs(g))))
9      a = float(min(amax, max(amin, a0)))
10     hist_f = [model.value(z)]; hist_gap = [projection_gap(z, g, lo, hi)]
11     converged = False
12     for it in range(1, maxit + 1):
13         w = project(z - a * g, lo, hi)          # 投影点
14         d = w - z                               # 投影弧方向
15         gd = float(np.dot(g, d))
16         fref = max(hist_f[-min(M, len(hist_f)):]) # 非单调参考值
17         lam, accepted = 1.0, False
18         for _ in range(lsearch_max):             # 回溯搜索 lam
19             fnew = model.value(z + lam * d)
20             if fnew <= fref + delta * lam * gd:   # 非单调 Armijo 条件
21                 accepted = True
22                 break
23             lam *= tau
24         if not accepted:                         # 兜底: 最速下降方向重试
25             d = -g / max(1.0, float(np.linalg.norm(g))); gd = float(np.dot(g, d))
26             lam, accepted = 1.0, False
27             for _ in range(lsearch_max):
28                 fnew = model.value(z + lam * d)
29                 if fnew <= fref + delta * lam * gd:
30                     accepted = True
31                     break
32             lam *= tau
33         if not accepted:
34             break
35         z_new = z + lam * d
36         g_new = model.gradient(z_new)
37         s, y_ = z_new - z, g_new - g

```

```

38     if float(np.linalg.norm(s)) > 1e-14:
39         a = bb_stepsize(s, y_, amin, amax, variant=bb_variant) # BB 步长
40     z, g = z_new, g_new
41     hist_f.append(model.value(z)); hist_gap.append(projection_gap(z, g, lo, hi))
42     if hist_gap[-1] <= tolP or float(np.max(np.abs(s))) <= tolX:
43         converged = True # 投影间隙 <= tolP: KKT
44     break
45 return dict(u=z, it=it, hist_f=np.array(hist_f), hist_gap=np.array(hist_gap),
46           converged=bool(converged))

```

程序 4 修正 PRP+ 共轭梯度法 (src/cg_solvers.py)

```

1 def nonlinear_cg(model, x0, beta="prp+", maxit=5000, tolG=1e-6, c1=1e-4, c2=0.1,
2   stop="rel", rel_tol=1e-6, safeguard=True, fallback=True):
3     """非线性共轭梯度 (强 Wolfe 线搜索), stop='rel' 为相对目标变化停止准则."""
4     x = np.asarray(x0, dtype=np.float64).copy()
5     g = model.gradient(x)
6     d = -g.copy()
7     f_cur = model.value(x)
8     counts = dict(negative_beta_count=0, beta_truncation_count=0,
9       descent_restart_count=0, line_search_fallback_count=0)
10    it = 0
11    for it in range(1, maxit + 1):
12        if float(np.max(np.abs(g))) <= tolG: # 梯度准则提前停止
13            break
14        alpha, ls_failed = strong_wolfe_alpha(model, x, d, g, c1, c2)
15        counts["line_search_fallback_count"] += int(ls_failed)
16        if alpha <= 0.0 or ls_failed: # 兜底: 负梯度重试
17            if not fallback:
18                break
19            d = -g.copy()
20            alpha, ls_failed = strong_wolfe_alpha(model, x, d, g, c1, c2)
21            counts["line_search_fallback_count"] += int(ls_failed)
22            if alpha <= 0.0:
23                break
24        x_new = x + alpha * d
25        g_new = model.gradient(x_new)
26        f_new = model.value(x_new)
27        if stop == "rel" and it > 1: # 相对目标变化停止
28            if abs(f_new - f_cur) / max(abs(f_cur), 1e-12) <= rel_tol:
29                x, g, f_cur = x_new, g_new, f_new
30            break
31        y_prev = g_new - g
32        if beta in ("prp", "prp+"): # beta = g'y / g'g
33            den = float(np.dot(g, g))
34            raw_beta = float(np.dot(g_new, y_prev)) / den if den > 1e-300 else 0.0
35        elif beta == "hs+": # HS 参数
36            den = float(np.dot(d, y_prev))
37            raw_beta = float(np.dot(g_new, y_prev)) / den if abs(den) > 1e-300 else 0.0
38        else: # FR / DY
39            raw_beta = None
40        if raw_beta is not None and raw_beta < 0.0: # 诊断: 负 beta / 截断
41            counts["negative_beta_count"] += 1
42            if beta in ("prp+", "hs+"):
43                counts["beta_truncation_count"] += 1
44        b = beta_rule(beta, g_new, g, d, y_prev)
45        d_new = -g_new + b * d
46        gd = float(np.dot(g_new, d_new))
47        if safeguard and gd >= -1e-7 * float(np.dot(g_new, g_new)):
48            d_new = -g_new.copy() # 下降性不足: 重启
49            counts["descent_restart_count"] += 1
50        x, g, d, f_cur = x_new, g_new, d_new, f_new
51    return dict(x=x, it=it, conv=True, **counts)

```

```

52
53 def cg_with_mu_continuation(model, x0, mu_seq=(1e-1, 1e-2, 1e-3),
54                             beta="prp+", maxit=2000, tolG=1e-6,
55                             stop="rel", rel_tol=1e-6, safeguard=True):
56     """mu 续延：由大到小逐段求解，每段以上一段解热启动（算法 2）。"""
57     x = np.asarray(x0, dtype=np.float64).copy()
58     it_sum, it_hist, conv_all = 0, [], True
59     for mu in mu_seq:
60         model.set_mu(float(mu))
61         res = nonlinear_cg(model, x, beta=beta, maxit=maxit, tolG=tolG,
62                             stop=stop, rel_tol=rel_tol, safeguard=safeguard)
63         x = res["x"]; it_sum += int(res["it"]); it_hist.append(int(res["it"]))
64         conv_all &= bool(res["conv"])
65     return dict(x=x, it_sum=it_sum, it_hist=it_hist, conv=conv_all)

```